

目录

- 1 项目概述
- 2 系统架构
- 3 玩法设计
- 4 核心实现
- 5 设计取舍与贡献
- 6 构建与总结

项目简介

- **Snake Terminal**: C++17 + raylib 的双人图形化贪吃蛇
- 架构: Scene 回调式单线程主循环 (参考 Godot / Unity 场景思路)
- 玩法可配置; 含动画状态机、虚影复活、自研音频与日志
- 构建: CMake + build.sh / build.bat; 配置: snake.cfg
- 依赖: raylib (子模块)、miniaudio、dr_mp3

目标

逻辑与表现解耦、场景可扩展, 在双人玩法之上做出完整工程闭环。

目录结构

src/

app.cpp

global.h

event/

config/

audio/

render/

utils/

scene/

game/

入口：切目录→CLI→配置→音频→主循环

跨场景状态 + AudioManager

Event / Input / Quit / SceneSwitch

Config + loader

AudioManager / Player / Loader

DrawList / Sprite

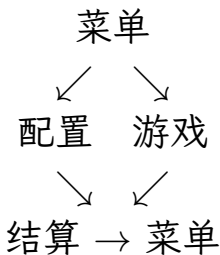
线程安全彩色日志

Menu / Config / Game / End

Snake + SnakeBody 动画状态机

场景流与生命周期

场景流



阶段	方法
进入	<code>on_enter()</code>
输入	<code>on_inputevent</code>
更新	<code>update(dt)</code>
渲染	<code>render()</code>
退出	<code>on_exit()</code>
切换	<code>is_finished / next</code>

主循环 (SceneManager::run)

单线程，每帧：

- ① 按键白名单 → 构造 InputEvent (Press / Release / repeat)
- ② handle_event: on_event 通用钩子 → dynamic_cast 分发
- ③ update(dt); 若 is_finished → switch_to
- ④ BeginDrawing → render() → EndDrawing

单线程约束

所有 raylib 调用 (InitWindow / LoadTexture / Draw*) 必须同线程，否则 OpenGL context 与 thread_local 出问题。

事件分发

```
void handle_event(Event& event) {  
    if (event.is_consumed()) return;  
    on_event(event);           // (1) 通用钩子, 可拦截  
    if (event.is_consumed()) return;  
    on_event_internal(event); // (2) dynamic_cast -> 具体类型  
}
```

子类只需覆盖需要的处理程序 (如 `on_input_event`), 用 `event.consume()` 阻止后续处理。

全局共享状态

- 分数: `last_score_player1/2`
- 玩家状态: `ALIVE / ON_WALL / ON_SELF / ON_PLAYER / STARVED`
- 结束原因 (高位不覆盖低位):
`FULL_BOARD > TIMEOUT > MANUAL > DEATH`
- 模式: `DEATHMATCH / TIMERACE`

结算显示:

- `FULL_BOARD` → `YOU'VE FILLED EVERYWHERE!!!`
- `TIMEOUT` → `TIME LIMIT REACHED`
- `MANUAL` → `GAME ENDED`
- `DEATH` → 逐玩家显示死因 (跳过 `ALIVE`)

双模式

模式	规则	死亡后
Deathmatch	撞墙 / 己 / 对方即终局	死亡方动画后消失；幸存者冻结 → 结算
Time Race	限时计分，可复活	切尾 <code>reborn_costs</code> ；虚影；切空则饿死

Tick 计时：两玩家独立 `tick_remain1_/2_` (`dt` 累积)。
Shift 按下时 `tick` 间隔减半；`check_collide` 按 `tick` 标志分别判定，
同时死亡则双方 `handle_death`。

操作

按键	作用
W/A/S/D	玩家 1 方向 / 死亡动画打断
I/J/K/L	玩家 2 方向 / 死亡动画打断
L-Shift / R-Shift	加速 (需 allowAcceleration)
ESC / Backspace	暂停 / 继续
T / Y (暂停时)	回菜单 / 手动结束
Shift+A/D (配置)	调整量 $\times 10$

配置 (snake.cfg)

- volume (0-100)
- allowAcceleration
- toroidalSpace 环面
- allowThroughOthers
- speed_factor (≥ 0.1)
- increasing_difficulty
- time_match_duration (0=inf)
- reborn_costs
- respawnInAdvance 虚影
- deathAnimInterruptThreshold

缺项用默认值补全，正常退出写回完整文件；读写失败仅打警告，不影响运行。

蛇逻辑与动画分离

Snake (纯逻辑)

- 移动、碰撞、得分
- 复活、尾部切除
- 虚影 / 可碰撞开关
- 不感知动画

SnakeBody + AnimateManager

- SnakeMove: 移动/加速
- SnakeDie: 逐节 X
- SnakeWaiting: 虚影/冻结

由 GameScene 驱动转换:

MOVE $\xrightarrow{\text{死亡}}$ DIE $\xrightarrow{\text{播完/打断}}$ WAITING $\xrightarrow{\text{按键}}$ MOVE

虚影复活与动画打断

Ghost (TIMERACE + respawnInAdvance):

- ❶ 死亡 → 原位播放 X 标记动画
- ❷ 动画完成 → 先切尾扣分, 再 `generate_ghost()` (半透明)
- ❸ 玩家确认位置后按键 → `deploy_from_ghost()` (不再重复扣分)

死亡动画打断 (`deathAnimInterruptThreshold`):

- -1: 关闭, 必须看完
- 0+: 前 N 节正常播, 之后按移动键跳过剩余

音频系统

共享设备 + 预解码 PCM + 回调混音

- AudioManager: 唯一 ma_device (48kHz 立体声 f32)
- 模板库不直接播; play_sound 独占, play_sfx 克隆入并发池
- AudioLoader: 加载时预解码 MP3 → float32 PCM
- mix(): 线性插值重采样 + 音量 + 声道转换
- 设备持续运行, 无声输出静音, 避免 ma_device_stop 死锁

对比 raylib Music: 主线程卡顿会导致欠载; 本方案音频线程独立于帧率。

日志系统

线程安全彩色日志，级别过滤 + 接管 raylib 日志：

级别	函数	颜色
Debug	logd	青
Info	log	绿
Warning	logw	黄
Error	loge	红

CLI: `./dist/snake --loglevel debug` (默认 Info)

启动时切到可执行文件目录；Windows 额外设 UTF-8 + 禁用 IME。

已知限制与取舍

- ① 纹理无引用计数 (raylib): 各 Sprite 独享 texture, 用完即卸, 安全但费显存
- ② 自研音频: 绕开 raylib Music 主线程依赖; 缺总线级实时混音, 预用 loudnorm 平衡
- ③ **Sprite** 职责偏多: 加载 / 纹理 / 变换 / 帧动画合一; 本规模下拆分收益不大
- ④ 状态机持裸指针: move 后需修复; 双角色不上完整 ECS
- ⑤ 物理帧 = 渲染帧: 无独立固定步; 碰撞嵌在 Snake 友元函数中

上游贡献

raylib ImageFromImage 边界错误

矩形校验用了严格比较 $>$ / $<$ 而非 $\geq$ / $\leq$ ，导致从 $(0,0)$ 裁剪或裁剪完整图像被错误拒绝。

已提交并合并：

<https://github.com/raysan5/raylib/pull/5979>

构建与运行

Linux / macOS

./build.sh

./build.sh --debug

./build.sh --build-raylib

./build.sh --clean

增量构建

Debug

含 raylib 重编译

Windows (MinGW)

./build.bat / ./build.bat --debug

./dist/snake --loglevel debug

总结

已完成

双人双模式；Scene/事件框架；可配置玩法；动画与虚影；自研音频与日志；配置持久化；上游 PR。

可继续

地图障碍 / 关卡；网络对战；纹理资源池；渲染职责拆分。

谢谢老师！ 欢迎提问